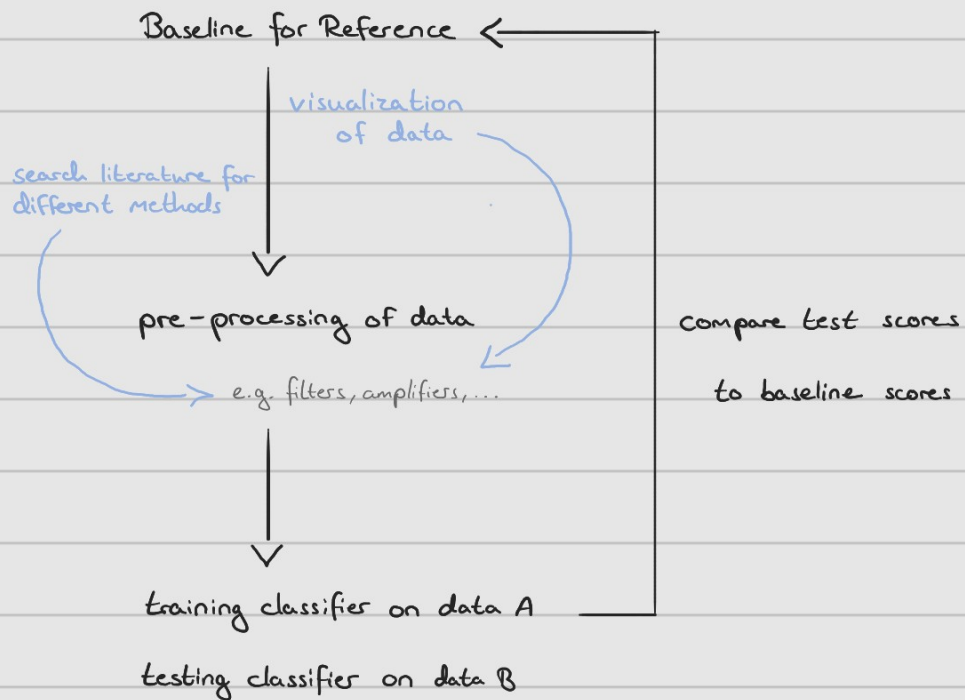


CAPSTONE - OVERVIEW



STEP 1: • Research Literature

↳ brainstorm methods for pre-processing

- familiarize with provided code and clear questions

e.g. What is the difference of testing between
"within-subject" and "new-subject"?

Within-Subject: Train and test on the same person, split by movement repetitions (e.g., train on repetitions 1–8, test on 9–10). This evaluates how accurately a model learns an individual user's specific muscle patterns.

New-Subject (Leave-One-Subject-Out / LOSO): Train on a group of subjects (e.g., Subjects 1–26) and test on a completely unseen user (Subject 27). This evaluates how well the model generalizes across different body builds, skin impedances, and electrode placements without requiring user-specific retraining.

CAPSTONE PROJECT CODE FILES

run_baseline.py

```
import sys
sys.path.insert(0, "src")

from emg_ninapro import EMGNinaproTrack

track = EMGNinaproTrack()
records = track.load("NinaPro_Mat", exercise="E1")
records = [r for r in records
           if r.group in {"S1", "S2", "S3", "S4", "S5"}]

results = track.evaluate_modes(records, cfg={"seed": 0})

for mode, result in results.items():
    print(mode, result["summary"])
    print("Pooled macro-F1:", result["macro_f1"])
    track.report(result)
```

=> running minimally pre-processed data to create reference

↳ compare with data performance, after applying more and better methods for...

adapter.py

- 7-module-track harness
- default classifiers
- Chapter 8 noise-filtering functions (can be copied and used in main script)

↳ ... filtering
... feature extraction

▽ Do NOT MODIFY ▽

report.py

↳ copy printed `rep["summary"]` outputs directly into

RESULTS.md

- ✓ free to edit
- ✗ off-limits code
- moderate editing

emg_ninapro.py

Imports and Constants (FS , WIN , STEP , N_CH)

- What it is/does: Imports required modules and defines baseline sampling/windowing parameters (100 Hz, 200 ms window, 150 ms step, 10 channels).
- Can you edit/play around here? YES — Modifying WIN (window size) and STEP (stride) directly alters temporal resolution, feature stability, and latency trade-offs.

```
from __future__ import annotations
import numpy as np

from adapter import TrackAdapter, TrackMeta, Recording, default_baseline, denoise

FS = 100.0
WIN = 20          # 200 ms window
STEP = 15         # 150 ms step
N_CH = 10
```



`_win_features(win, fs=FS)`

- **What it is/does:** Computes 5 time/frequency-domain features (MAV, RMS, Waveform Length, Variance, Mean Frequency) for each channel across a single window.
- **Can you edit/play around here? YES (PRIMARY LOCATION)** — This is the core feature engineering function where you can add new DSP features (e.g., zero crossings, slope sign changes, autoregressive coefficients, band powers).

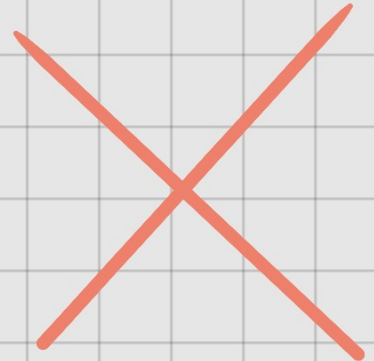
```
def _win_features(win, fs=FS):
    """Amplitude/energy features per channel from a [win_samples, n_ch] window."""
    feats = []
    for ch in range(win.shape[1]):
        x = win[:, ch]
        mav = float(np.mean(np.abs(x)))
        rms = float(np.sqrt(np.mean(x ** 2)))
        wl = float(np.sum(np.abs(np.diff(x))))          # waveform length
        var = float(np.var(x))
        X = np.abs(np.fft.rfft(x - x.mean())) + 1e-12   # coarse mean frequency
        f = np.fft.rfftfreq(len(x), 1 / fs)
        mnf = float(np.sum(f * X) / np.sum(X))
        feats += [mav, rms, wl, var, mnf]
    return feats
```



`EMGNinaproTrack` Class Definition & `meta` Attribute

- **What it is/does:** Initializes the dataset track class and sets track metadata (dataset details, evaluation targets, gesture class labels).
- **Can you edit/play around here? NO** — Fixed dataset metadata required by the benchmark pipeline framework.

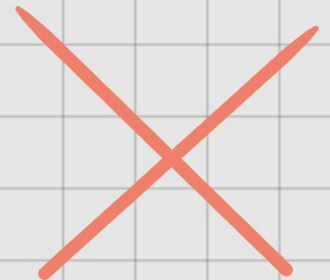
```
class EMGNinaproTrack(TrackAdapter):
    meta = TrackMeta(
        name="EMG gesture",
        dataset="Ninapro DB1 (surface EMG, exercise E1)",
        dataset_version="1.0",
        license="CC BY-NC-ND [ ] cite Atzori et al. (2014)",
        citation="Atzori M, et al. Sci Data (2014). Ninapro DB1.",
        url="https://ninapro.hevs.ch/instructions/DB1.html",
        signals=["10-channel surface EMG envelope @ 100 Hz"],
        task_type="multiclass",
        classes=[f"g{i:02d}" for i in range(1, 13)],      # 12 finger movements (E1)
        split_unit="subject",
        default_metrics=["macro_f1", "cohens_kappa"],
        smoke_test_records=["S1", "S2", "S3"],
        expected_runtime="~3 [ ] min on a subject subset (Colab, CPU)",
        dsp_focus="windowing, MAV/RMS/waveform-length/variance + mean-frequency per channel",
        difficulty=3,
        eval_modes=("within-subject", "new-subject"),
    )
```



`CFG_KEY_HINTS` Dictionary

- **What it is/does:** Stores instructional error messages explaining why certain configuration options are restricted or handled specially.
- **Can you edit/play around here? NO** — Static framework text for error reporting.

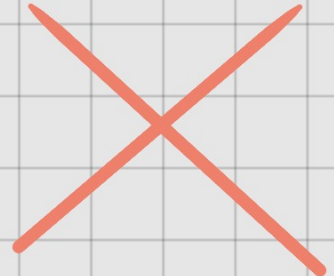
```
CFG_KEY_HINTS = {
    "preprocess": "EMG's preprocess() is an identity stub. The stage-2 move that attacks "
    "track's cross-subject gap is per-channel normalisation WITHIN a subject "
    "(i.e. a calibration procedure you must then declare) [ ] write it, then "
    "register its knobs with track.declare_cfg_keys(...).",
    "spectral_method": "EMG's features are MAV / RMS / waveform length / variance plus a "
    "frequency taken from an rFFT magnitude spectrum [ ] nothing here "
    "integrates a PSD, so there is no spectral estimator to choose.",
}
```



smoke(...) Method

- What it is/does: Synthesizes multi-subject sEMG data with noise, electrode shifts, and cross-channel bleed to enable offline testing without full dataset downloads.
- Can you edit/play around here? NO — Internal test data generator; editing this does not improve real-data processing or classification performance.

```
def smoke(self, n_subjects=5, n_reps=6, win_per_rep=4, electrode_shift=2,
          bleed=0.5, rep_sd=0.40, win_noise=0.20, gain_sd=0.35, seed0=4000):
    recs = []
    gestures = self.meta.classes
    # a fixed, distinct 3-channel activation template per gesture (shared by all subjects)
    templates = []
    for gi in range(len(gestures)):
        trng = np.random.default_rng(9000 + gi)
        tv = np.zeros(N_CH)
        tv[trng.choice(N_CH, size=3, replace=False)] = trng.uniform(0.3, 0.6, size=3)
        templates.append(tv)
    for s in range(n_subjects):
        rng = np.random.default_rng(seed0 + s)
        shift = int(rng.integers(-electrode_shift, electrode_shift + 1))
        # crosstalk / volume conduction onto neighbouring electrodes, per subject
        M = np.eye(N_CH)
        b = bleed * rng.uniform(0.5, 1.5)
        for i in range(N_CH):
            M[i, (i + 1) % N_CH] += b * rng.uniform(0.3, 1.0)
            M[i, (i - 1) % N_CH] += b * rng.uniform(0.3, 1.0)
        M /= M.sum(axis=1, keepdims=True)
        ch_gain = np.exp(gain_sd * rng.standard_normal(N_CH))
        wins, labs, reps = [], [], []
        for gi, g in enumerate(gestures):
            base = np.roll(templates[gi] @ M, shift) * ch_gain
            for rep in range(1, n_reps + 1):
                # one contraction = one shared state; its windows are correlated
                rep_v = base * np.exp(rep_sd * rng.standard_normal(N_CH))
                for _ in range(win_per_rep):
                    w = np.abs(rep_v[None, :]) * (1 + win_noise * rng.standard_normal((WIN, N_CH)))
                    w += 0.6 * win_noise * rng.standard_normal((WIN, N_CH))
                    wins.append(w); labs.append(g); reps.append(rep)
            recs.append(Recording(group=f"s{s + 1}", fs=FS,
                                epochs={"emg": np.array(wins)},
                                labels=np.array(labs), meta={"rep": np.array(reps)}))
    return recs
```



preprocess(self, rec: Recording, cfg=None)

- What it is/does: Acts as an identity pass-through function where input data is returned unchanged by default.
- Can you edit/play around here? YES (PRIMARY LOCATION) — The main entry point for signal processing improvements such as per-subject normalizations, extra envelope filtering, and channel transformations.

```
# ---- module 2: PREPROCESSING (a stub you fill in but read the warning) ----
def preprocess(self, rec: Recording, _cfg=None) -> Recording:
    return rec
```



extract_features(self, rec: Recording, cfg=None)

- What it is/does: Loops over all epoch windows in a recording, invokes `_win_features` on each, and returns feature matrices with corresponding gesture labels.
- Can you edit/play around here? YES — Useful if you need to pass additional metadata into feature extraction or reshape the feature matrix for sequence/deep learning models.

```
def extract_features(self, rec: Recording, _cfg=None):
    X = [_win_features(w, rec.fs) for w in rec.epochs["emg"]]
    return np.array(X), np.asarray(rec.labels), rec.group
```



baseline(self, cfg=None)

- **What it is/does:** Builds and configures the default Random Forest classifier alongside class-imbalance options like SMOTE or class reweighting.
- **Can you edit/play around here? YES** — You can adjust machine learning hyper-parameters, change default imbalance handling strategies, or swap out model defaults.

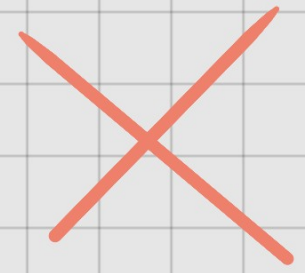
```
def baseline(self, cfg=None):  
    cfg = self._cfg(cfg)  
    return default_baseline(seed=int(cfg.get("seed", 0)),  
                           imbalance=cfg.get("parameter") cfg: dict,  
                           threshold=float(cfg.get("threshold", 0.5)),  
                           smote_k=int(cfg.get("smote_k", 5)))
```



_build(self, recs, cfg=None)

- **What it is/does:** Iterates over multiple subject recordings to construct concatenated feature matrices (X) and group arrays (y , subj, rep).
- **Can you edit/play around here? NO** — Internal pipeline helper for dataset aggregation.

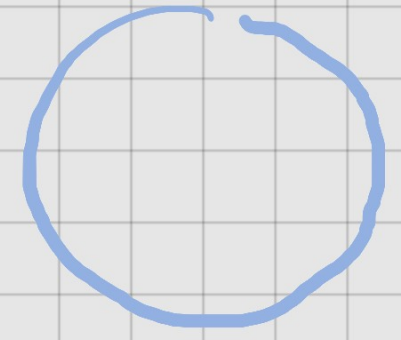
```
# ---- both evaluation modes ----  
def _build(self, recs, cfg=None):  
    Xs, ys, subj, rep = [], [], [], []  
    for r in recs:  
        X, y, g = self._features_for(r, cfg)  
        Xs.append(X); ys.append(y)  
        subj.append(np.array([g] * len(y)))  
        rep.append(np.asarray(r.meta.get("rep", np.zeros(len(y)))))  
    return (np.vstack(Xs), np.concatenate(ys),  
            np.concatenate(subj), np.concatenate(rep))
```



evaluate_modes(self, recs, cfg=None)

- **What it is/does:** Evaluates model performance across both within-subject (leave-repetition-out) and new-subject (LOSO) cross-validation protocols.
- **Can you edit/play around here? YES (MODERATE)** — You can insert custom feature selection algorithms (`select_features`) or specialized classifier models directly into the cross-validation fold loops.

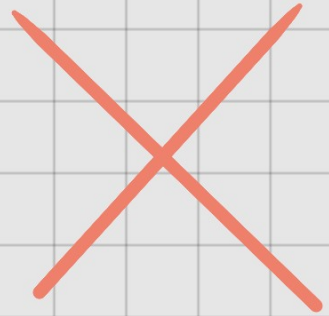
```
def evaluate_modes(self, recs, cfg=None):  
    """Both required modes, each with its PER-SUBJECT spread (16.3): the gap  
    between them is the finding, and a pooled pair of numbers hides which  
    subjects drive it."""  
    from sklearn.base import clone  
    from sklearn.model_selection import GroupKFold  
    cfg = self._cfg(cfg)  
    X, y, subj, rep = self._build(recs, cfg)  
    new_subject = self.evaluate(X, y, subj, cfg=cfg) # LOSO/GroupKFold on subject  
    yt, yp, gg, ff = [], [], [], []  
    k = 0  
    for s in sorted(set(subj.tolist())):  
        # within subject: leave-reps-o  
        m = subj == s  
        Xs, ys, rs = X[m], y[m], rep[m]  
        n_rep = len(set(rs.tolist()))  
        if n_rep < 2 or len(set(ys.tolist())) < 2:  
            continue  
        for tr, te in GroupKFold(n_splits=min(5, n_rep)).split(Xs, ys, rs):  
            sel = self.select_features(Xs[tr], ys[tr], cfg) # module 4, inside the fold  
            clf = clone(self._baseline(cfg)).fit(sel.transform(Xs[tr]), ys[tr])  
            yp.extend(clf.predict(sel.transform(Xs[te]))); yt.extend(ys[te])  
            gg.extend([s] * len(te)); ff.extend([k] * len(te))  
            k += 1  
    within = self._make_report(yt, yp, groups=gg, folds=ff,  
                              split_unit="repetition (within subject)",  
                              n_groups=len(set(subj.tolist())), group_unit="subject")  
    return {"within_subject": within, "new_subject": new_subject}
```



```
download(self, cache_dir, subset=range(1, 6))
```

- **What it is/does:** Downloads and unpacks Ninapro DB1 .zip subject files from the remote server to local cache storage.
- **Can you edit/play around here? NO** — Basic file download I/O handler.

```
def download(self, cache_dir, subset=range(1, 6)):  
    """Download + extract Ninapro DB1 subject zips (s1.zip ...) into cache_dir.  
    Needs network. ~20 MB/subject. `subset` is 1-based subject ids (1..27)."""  
    import os, io, zipfile, urllib.request  
    os.makedirs(cache_dir, exist_ok=True)  
    for s in subset:  
        marker = os.path.join(cache_dir, f"s{s}_A1_E1.mat")  
        if os.path.exists(marker):  
            continue  
        blob = urllib.request.urlopen(f"{self._BASE}s{s}.zip", timeout=180).read()  
        with zipfile.ZipFile(io.BytesIO(blob)) as z:  
            z.extractall(cache_dir)  
    print(f"[emg_ninapro] ready in {cache_dir}")  
    return cache_dir
```



```
load(self, cache_dir, exercise="E1")
```

- **What it is/does:** Reads MATLAB .mat files, slices continuous sEMG into sliding windows, and filters out non-gesture or boundary samples.
- **Can you edit/play around here? YES** — Adjusts raw signal segmentation criteria, resting-state filtering, or target gesture exercise selections.

```
def load(self, cache_dir, exercise="E1"):  
    """Read Ninapro DB1 `*_{exercise}.mat` files, window the sEMG within each  
    (movement, repetition) segment, label by movement. Groups by SUBJECT."""  
    import os, glob, re  
    from scipy.io import loadmat  
    gest_set = {int(g[1:]) for g in self.meta.classes}  
    recs = []  
    mats = sorted(glob.glob(os.path.join(cache_dir, "****", f"*_{exercise}.mat")), recursive)  
    for mat in mats:  
        m = re.search(r"s(\d+)_", os.path.basename(mat))  
        if not m:  
            continue  
        subj = f"s{m.group(1)}"  
        d = loadmat(mat)  
        emg = np.asarray(d["emg"], float) # [N, 10]  
        lab = np.asarray(d["restimulus"]).ravel()  
        rep = np.asarray(d["rerepetition"]).ravel()  
        wins, labs, reps = [], [], []  
        for s in range(0, len(emg) - WIN + 1, STEP):  
            wl = lab[s:s + WIN]; wr = rep[s:s + WIN]  
            g = int(wl[0])  
            if g in gest_set and (wl == g).all() and (wr == wr[0]).all() and wr[0] > 0:  
                wins.append(emg[s:s + WIN]); labs.append(f"g{g:02d}"); reps.append(int(wr[0]))  
        if wins:  
            recs.append(Recording(group=subj, fs=FS,  
                                epochs={"emg": np.array(wins)},  
                                labels=np.array(labs), meta={"rep": np.array(reps)}))  
    return recs
```



```
if __name__ == "__main__": Execution Block
```

- **What it is/does:** Runs a test script that executes smoke() synthetic benchmarking and prints macro-F1 and Cohen's Kappa evaluation metrics.
- **Can you edit/play around here? YES** — The test execution script where you trigger pipeline runs, evaluate real vs. synthetic data, and print performance reports.

```
if __name__ == "__main__":  
    t = EMGNinaproTrack()  
    print(t.dataset_card())  
    modes = t.evaluate_modes(t.smoke())  
    from sklearn.metrics import f1_score  
    for name, rep in modes.items():  
        print("SMOKE %-15s macroF1=%.3f kappa=%.3f (%s)" % (  
            name, f1_score(rep["y_true"], rep["y_pred"], average="macro"),  
            rep["cohens_kappa"], rep["split_unit"]))  
    print(" spread:", rep["summary"]) # the per-subject range, not just the pool
```



